

MagicCode Super-Resolution Software Integration Guide v1.1.0

Document Version: v1.1.0

1. Scope

This guide explains how to integrate MagicCode Super-Resolution Software v1.1.0 into native applications, Android/iOS applications, Unity projects, and Unreal Engine projects.

2. Native C/C++ Integration

- 1 Copy mc_interface.h into your include path.
- 2 Copy the matching libmagic_sr library into your platform library path.
- 3 Link platform frameworks or system libraries required by the selected backend, such as Metal/CoreVideo/Foundation on iOS or Vulkan/OpenGL ES components on Android where applicable.
- 4 Package the model file with your application or place it in a readable runtime path.
- 5 Initialize one handle per stream or processing configuration.
- 6 Process frames and query status during validation.
- 7 Release the handle during shutdown.

3. Backend and Input Type Matching

Backend	Typical Platform	Input Type	Notes
---------	------------------	------------	-------

X86			
-----	--	--	--

Windows/macOS desktop CPU			
---------------------------	--	--	--

INPUT_BUFFER

Requires supported x86_64 CPU features.

NEON

Android/iOS ARM CPU

INPUT_BUFFER

Requires ARM64 NEON support.

METAL

iOS/iPadOS/macOS

Texture

Requires compatible Metal runtime and model.

OPENGL

Desktop GPU where supported

Texture

Requires compatible OpenGL context.

OPENGLES

Android mobile GPU

Texture

Requires compatible OpenGL ES context and model.

VULKAN

Android mobile GPU

Texture

Requires compatible Vulkan device/driver and model.

4. Android Notes

- Recommended ABI: arm64-v8a.
- Ensure model files are copied to assets or another path readable by native code.
- Validate on physical devices because emulator graphics behavior may differ from production devices.
- For Unity Android, place libmagic_sr_unity.so under Assets/Plugins/Android/arm64-v8a/.

- For Unreal Android, verify libmagic_sr.a exists at the path expected by MagicSR.Build.cs.

5. iOS/iPadOS Notes

- Use ARM64 device builds for runtime validation.
- Link required Apple frameworks for Metal-based paths.
- Package model files into the app bundle or copy them into a readable sandbox path.
- For Unity iOS, include libmagic_sr_unity_ios.a in the generated Xcode target.
- For Unreal iOS, ensure the UE version, Xcode version, and Apple SDK version are mutually compatible.

6. Unity Integration

- 1 Build or obtain Unity native plugin artifacts.
- 2 Copy C# wrappers into the Unity project.
- 3 Set model paths and target textures/images.
- 4 Use MagicSRSession.Create or CreateEx.
- 5 Use Process for buffer input or ProcessTexture for texture input.
- 6 Call SetParam and query status during validation.
- 7 Call Destroy when the session is no longer needed.

7. Unreal Engine Integration

- 1 Copy plugin/ue/MagicSR into the target project's Plugins directory.
- 2 Enable the plugin and regenerate project files if required.

3 Use Blueprint nodes or C++ APIs: GetVersion, CreateSession, ProcessY8, SetParam, QueryStatus, and DestroySession.

4 Package model files into a runtime-readable location.

5 Run smoke validation on device and inspect status/error codes.

8. Integration Code Examples

8.1 Native C/C++ Buffer Input

This example shows the minimum native integration flow for CPU buffer input. Replace model paths, image buffers, backend, and dimensions with your production values.

```
input_param_t param;
memset(&param, 0, sizeof(param));
param.input_type = INPUT_BUFFER;
strncpy(param.model_path,
"/path/to/magic_neon_highspeed_params.bin",
sizeof(param.model_path) - 1);
param.width = 1280;
param.height = 720;
param.scaler_factor = 2.0f;
param.alg_mode = HIGH_SPEED_MODE;
param.num_threads = 4;
param.log_level = MAGIC_LOG_INFO;
param.backend = MAGIC_BACKEND_NEON;
```

```

void *handle = MC_Init(&param);
if (handle != NULL) {
    int ret = MC_Process(handle, input_y_buffer, output_y_buffer);
    output_status_params_t status;
    memset(&status, 0, sizeof(status));
    MC_Control(handle, QUERY_STATUS, NULL, &status);
    MC_Uninit(handle);
}

```

8.2 GPU Texture Backend Skeleton

GPU integrations must provide valid texture handles created in the same graphics environment expected by the selected backend. The exact texture handle type is platform/backend-specific.

```

input_param_t param;
memset(&param, 0, sizeof(param));

param.input_type = INPUT_TEXTURE_R8Unorm;           /* or
INPUT_TEXTURE_RGB8Unorm */

param.backend = MAGIC_BACKEND_VULKAN;              /* use
MAGIC_BACKEND_METAL on iOS/macOS */

param.alg_mode = HIGH_SPEED_MODE;

param.scaler_factor = 2.0f;

param.width = texture_width;

param.height = texture_height;

param.num_threads = 1;

strncpy(param.model_path, model_path, sizeof(param.model_path))

```

```
- 1);
```

```
void *handle = MC_Init(&param);
```

```
if (handle != NULL) {
```

```
    int    ret    =    MC_Process(handle,    input_texture_handle,  
output_texture_handle);
```

```
    MC_Uninit(handle);
```

```
}
```

8.3 Unity C# Skeleton

Unity projects should use the provided C# wrapper and native plugin artifacts. The exact enum names may vary slightly with wrapper version; keep the wrapper and native library from the same release package.

```
using MagicSR;
```

```
MagicSRSession session = new MagicSRSession();
```

```
bool ok = session.CreateEx(
```

```
    modelPath,
```

```
    width,
```

```
    height,
```

```
    2.0f,
```

```
    MagicSRAlgMode.HighSpeed,
```

```
    4,
```

```
    MagicSRInputType.Buffer,
```

```
    MagicSRBackend.Vulkan);
```

```
if (ok) {  
    int ret = session.Process(inputY, outputY);  
    session.Destroy();  
}
```

8.4 Unreal Engine Blueprint/C++ Flow

Unreal Engine projects should use the plugin session lifecycle exposed by the MagicSR Blueprint/C++ bridge.

// Blueprint flow:

// 1. GetVersion

// 2. CreateSession(ModelPath, Width, Height, Scale, AlgMode,
NumThreads)

// 3. ProcessY8(SessionId, InputY, OutputY)

// 4. QueryStatus(SessionId)

// 5. DestroySession(SessionId)

// C++ integrations should follow the same session lifecycle and
ensure

// model files are packaged into a runtime-readable path.

8.5 Example Integration Notes

- Do not mix libraries, headers, wrappers, or model files from different MagicCode versions.
- Always validate output dimensions through status query during integration testing.
- Package only model files required by the selected platform, backend, mode, and scale.
- For real-time pipelines, reuse sessions where practical

instead of repeatedly initializing and uninitializing.

9. Validation Checklist

- Version string returns v1.1.0.
- Initialization succeeds with expected backend, mode, scale, and model.
- Output width and height match expected scale.
- Output is not all-zero and visual quality is acceptable for the target scenario.
- Error code is zero after processing.
- Resources are released without leak or double-free warnings.